

Q1. For every Page table entry in the print_pgtbl output, explain what it logically contains and what its permission bits are.

Flag	Meaning	Bit Position	Hex Value
V	Valid	0	0x1
R	Read	1	0x2
W	Write	2	0x4
X	Execute	3	0x8
U	User	4	0x10
G	Global	5	0x20
A	Accessed	6	0x40
D	Dirty	7	0x80

- Va : virtual address
 - PTE: the raw 64-bit PTE value
 - Pa: physical address extracted from the PTE
 - Perm: the low permission bits of the PTE
-
- va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
 - 0101 1011
 - 0x5B: 0x40 + 0x10 + 0x08 + 0x02 + 0x01
So **V, R, X, U, A** (no W, no D, no G).
 - This is the user text page (program code). It is readable, executable, and user-accessible. It has been accessed. It is not writable and not dirty.

 - va 0x1000 pte 0x21FC7C17 pa 0x87F1F000 perm 0x17:
 - 0001 0111
 - 0x17 = 0x10 + 0x04 + 0x02 + 0x01 → **V, R, W, U**
 - This is a user data page. It is readable, writable, and user-accessible. It has not been accessed yet (A=0) and is not dirty (D=0).

 - va 0x2000 pte 0x21FC7807 pa 0x87F1E000 perm 0x7
 - 0000 0111
 - 0x7 = 0x4 + 0x2 + 0x1 → **V, R, W**
 - This is the stack guard page. It is mapped, but the U bit is clear, so it is not user-accessible. It is readable and writable only in supervisor mode. If the user stack grows into this page, a page fault occurs, catching stack overflow.

 - va 0x3000 pte 0x21FC74D7 pa 0x87F1D000 perm 0xD7
 - 1101 0111
 - 0xD7 = 0x80 + 0x40 + 0x10 + 0x04 + 0x02 + 0x01 → **V, R, W, U, A, D**
 - This is the user stack page. It is readable, writable, user-accessible, and has already been accessed and written to (A=1, D=1).

- va 0x4000 pte 0x0 pa 0x0 perm 0x0
 - 0000 0000
 - These are unmapped pages. They are not valid. They represent free virtual address space where the heap could grow via sbrk.
 - The following fall in the same category.
 - va 0x5000 pte 0x0 pa 0x0 perm 0x0
 - va 0x6000 pte 0x0 pa 0x0 perm 0x0
 - va 0x7000 pte 0x0 pa 0x0 perm 0x0
 - va 0x8000 pte 0x0 pa 0x0 perm 0x0
 - va 0x9000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
 - va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0

- va 0x3FFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
 - 1100 0111
 - $0xC7 = 0x80 + 0x40 + 0x4 + 0x2 + 0x1 \rightarrow \mathbf{V, R, W, A, D}$
 - This is the trapframe. It is used by the kernel to save user registers when a trap occurs. It is supervisor read-write (no U), and it has been accessed and written.

- va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
 - 0100 1011
 - $0x4B = 0x40 + 0x8 + 0x2 + 0x1 \rightarrow \mathbf{V, R, X, A}$
 - This is the trampoline. It contains the code that switches between user and kernel mode. It is supervisor read-execute (no U, no W), and it has been accessed.

```
arwam@Arwa: /mnt/c/Users/arwam$ cd xv6-labs-2026
git fetch
git checkout pgtbl
make clean
branch 'pgtbl' set up to track 'origin/pgtbl'.
Switched to a new branch 'pgtbl'
rm -rf *.tex *.dvi *.idx *.aux *.log *.ind *.ilg *.dSYM *.zip *.pcap \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel.fs.img \
mkfs/mkfs.gdbinit \
user/usys.S \
user/cat user/echo user/forktest user/grep user/init user/kill user/ln user/ls user/mkdir user/rm user/sh user/stressfs user/usertests user/gr
ind user/wc user/zombie user/logstress user/forphan user/dorphan user/_sync user/_pgtbltest
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ cd ~/xv6-labs-2026
git remote add myrepo https://github.com/Arwa-Mahmood/os_xv6_labs.git
git push myrepo pgtbl:pgtbl-lab
-bash: cd: /home/arwam/xv6-labs-2026: No such file or directory
Enumerating objects: 42, done.
Counting objects: 100% (42/42), done.
Delta compression using up to 12 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (25/25), 15.59 MiB | 760.00 KiB/s, done.
Total 25 (delta 15), reused 9 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (15/15), completed with 15 local objects.
remote:
remote: Create a pull request for 'pgtbl-lab' on GitHub by visiting:
remote: https://github.com/Arwa-Mahmood/os_xv6_labs/pull/new/pgtbl-lab
remote:
To https://github.com/Arwa-Mahmood/os_xv6_labs.git
 * [new branch] pgtbl -> pgtbl-lab
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ make qemu
mv -f fs.img fs.img.bk
mv: cannot stat 'fs.img': No such file or directory
make: [Makefile:297: newfs.img] Error 1 (ignored)
riscv64-unknown-elf-gcc -march=rv64gc -g -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -c -o kernel/entry.o kernel/entry.S
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=.
-D__PGTBL__ -D__LAB_PGTBL__ -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-b
uiltin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-log -fno-builtin-bzero -fno-builtin-exit -fno-bu
iltin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-
protector -fno-pie -no-pie -c -o kernel/kalloc.o kernel/kalloc.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=
```

+ ... X
1631 - Arwa Mahmood

```
arwam@Arwa: /mnt/c/Users/arwam$ cd xv6-labs-2026
riscv64-unknown-elf-objdump -S user/pgtbltest > user/pgtbltest.asm
riscv64-unknown-elf-objdump -t user/pgtbltest | sed '1,/SYMBOL TABLE/d; s/ .* //; /$/d' > user/pgtbltest.sym
mkfs/mkfs fs.img README user/cat user/echo user/forktest user/grep user/init user/kill user/ln user/ls user/mkdir user/rm user/sh user/stressfs user/_usertes
ts user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_pgtbltest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 102 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,
id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C17 pa 0x87F1F000 perm 0x17
va 0x2000 pte 0x21FC7807 pa 0x87F1E000 perm 0x7
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
vmprint_test starting
vmprint_test: done
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=3
$
```

+ ... X
1631 - Arwa Mahmood

Q2. Walk and print a two-level page table (easy)

```
arwam@Arwa: /mnt/c/Users/ \ + v
va 0x3000 pte 0x21FC74D7 pa 0x87F1D000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
vmprint_test starting
vmprint_test: done
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=3
$ exit
exec exit failed
$ shutdown
exec shutdown failed
$ QEMU: Terminated
grep -n "vmprint" kernel/*.c kernel/defs.harwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ grep -n "vmprint" kernel/*.c kernel/defs.h
kernel/syscall.c:118:extern uint64 sys_vmprint(void);
kernel/syscall.c:160: [SYS_vmprint] = sys_vmprint,
kernel/sysproc.c:99:sys_vmprint(void)
kernel/sysproc.c:104: vmprint(p->pagetable);
kernel/vm.c:152:vmprint(pagetable_t pagetable)
kernel/defs.h:191:void vmprint(pagetable_t);
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$
```

```
arwam@Arwa: /mnt/c/Users/ \ + v
#if defined(LAB_PGTBL) || defined(SOL_MMAP) || defined(SOL_COW)
void
vmprint(pagetable_t pagetable)
{
    // your code here
}
#endif

// add a mapping to the kernel page table.
// only used when booting.
// does not flush TLB or enable paging.
void
kvmmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm)
{
    if (mappages(kpgtbl, va, sz, pa, perm) != 0)
        panic("kvmmmap");
}

// Create PTEs for virtual addresses starting at va that refer to
// physical addresses starting at pa.
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ grep -n "printf\|printk" kernel/defs.h
83:// printk.c
84:int printk(char*, ...) __attribute__((format(printf, 1, 2)));
86:void printkinit(void);
225:// sprintf.c
226:int snprintf(char*, unsigned long, const char*, ...);
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ nano vm.c
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ nano vm.c
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ kernel
kernel: command not found
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ cd kernel
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026/kernel$ nano vm.c
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026/kernel$ nano vm.c
```

31631 - Arwa Mahmood

```
arwam@Arwa: /mnt/c/Users/ X + v
pa = PTE2PA(*pte);
return pa;
}

#if defined(LAB_PGTBL) || defined(SOL_MMAP) || defined(SOL_COW)
static void
vmprint_walk(pagetable_t pt, int level, uint64 vabase)
{
    int shift = 12 + 9 * level;

    for(int i = 0; i < 512; i++){
        pte_t pte = pt[i];
        if(!(pte & PTE_V))
            continue;

        uint64 va = vabase | ((uint64)i << shift);

        for(int d = 0; d < 3 - level; d++){
            printf(" ..");
            printf("%p: pte %p pa %p ", va, pte, PTE2PA(pte));

            if(pte & PTE_R) printf("R");
            if(pte & PTE_W) printf("W");
            if(pte & PTE_X) printf("X");
            if(pte & PTE_U) printf("U");
            printf("\n");

            if((pte & (PTE_R | PTE_W | PTE_X)) == 0)
                vmprint_walk((pagetable_t)PTE2PA(pte), level - 1, va);
        }
    }

    void
    vmprint(pagetable_t pagetable)
    {
        printf("page table %p\n", pagetable);
        vmprint_walk(pagetable, 2, 0);
    }
}

30°C
Mostly cloudy
11:24 am
05/10/2026
```

```
arwam@Arwa: /mnt/c/Users/ X + v
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$ grep -n "printf" kernel/vm.c
make qemu
mv -f fs.img fs.img.bk
mv: cannot stat 'fs.img': No such file or directory
make: [Makefile:297: newfs.img] Error 1 (ignored)
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -DSOL_PGTBL -DLAB_PGTBL -march=xv64gc -std=gnu99 -MD -mcmmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-builtin-stack-protector -fno-pie -no-pie -c -o kernel/vm.o kernel/vm.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/stats.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ / / /; /.*$/d' > kernel/kernel.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_pgtbltest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1032 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

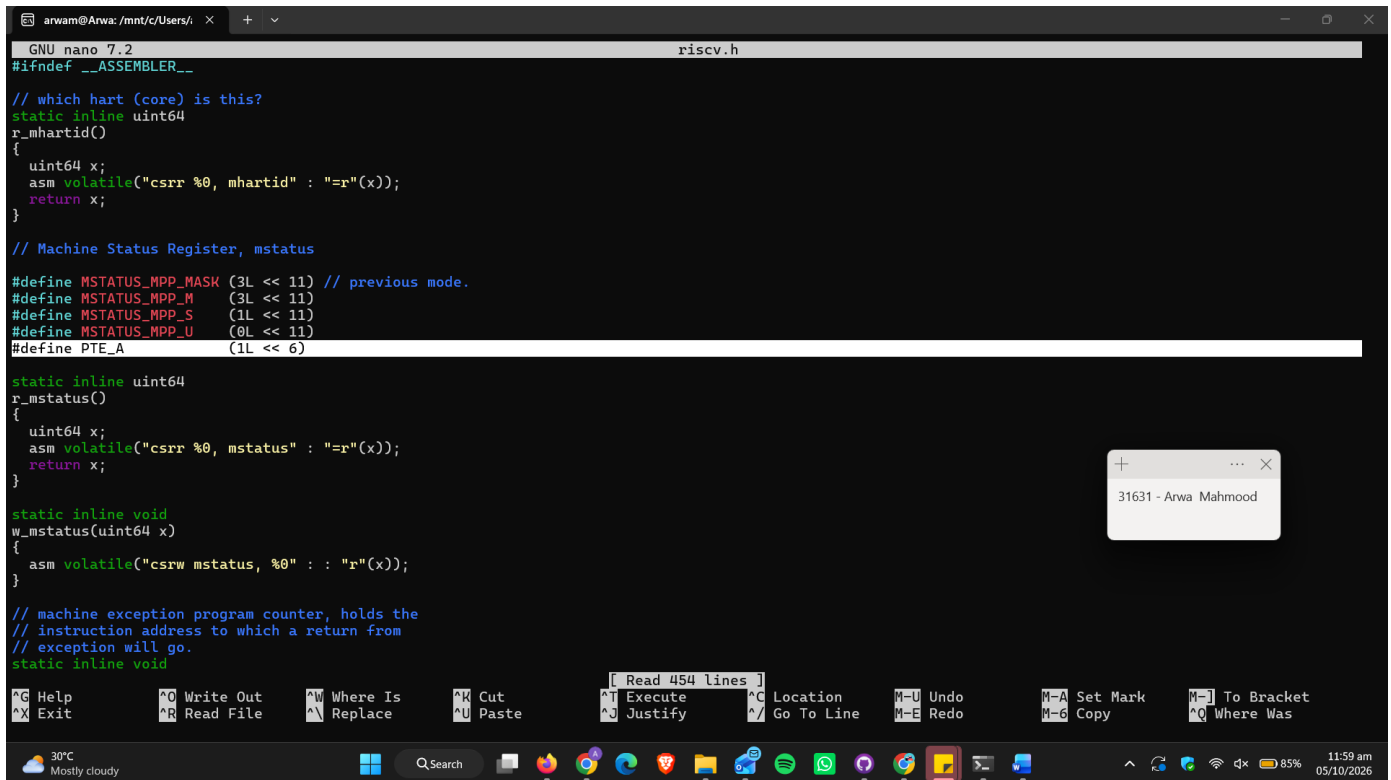
hart 1 starting
hart 2 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C17 pa 0x87F1F000 perm 0x17
va 0x2000 pte 0x21FC7807 pa 0x87F1E000 perm 0x7
va 0x3000 pte 0x21FC74D7 pa 0x87F1D000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0

30°C
Mostly cloudy
11:25 am
05/10/2026
```

```
arwam@Arwa: /mnt/c/Users/ / x + v
hart 2 starting
init: starting sh
$ pgtbltest
print_pttbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C17 pa 0x87F1F000 perm 0x17
va 0x2000 pte 0x21FC7807 pa 0x87F1E000 perm 0x7
va 0x3000 pte 0x21FC74D7 pa 0x87F1D000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pttbl: OK
vmprint_test starting
page table 0x0000000087f25000
..0x0000000000000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0x0000000000000000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. ..0x0000000000000000: pte 0x0000000021fc885b pa 0x0000000087f22000 RXU
.. ..0x00000000000001000: pte 0x0000000021fc7c17 pa 0x0000000087f1f000 RWU
.. ..0x00000000000002000: pte 0x0000000021fc7807 pa 0x0000000087f1e000 RW
.. ..0x00000000000003000: pte 0x0000000021fc74d7 pa 0x0000000087fd000 RWU
..0x00000003fc000000: pte 0x0000000021fc9001 pa 0x0000000087f24000
.. ..0x00000003ffffe0000: pte 0x0000000021fc8c01 pa 0x0000000087f23000
.. ..0x00000003fffffe000: pte 0x0000000021fd08c7 pa 0x0000000087f42000 RW
.. ..0x00000003ffffff000: pte 0x000000002000184b pa 0x0000000080006000 RX
vmprint_test: done
pgaccess_test starting
pgtbltest: pgaccess_test failed: incorrect access bits set, pid=3
$
```

31631 - Arwa Mahmood

Q3. Detect which pages have been accessed (easy)



The screenshot shows a terminal window with the nano editor open to the file `riscv.h`. The file contains RISC-V assembly macros for reading and writing machine status registers. A small notification bubble in the top right corner reads "31631 - Arwa Mahmood". The terminal's status bar at the bottom shows the temperature as 30°C, weather as "Mostly cloudy", and the date/time as 11:59 am on 05/10/2026.

```
GNU nano 7.2 riscv.h
#ifndef __ASSEMBLER__

// which hart (core) is this?
static inline uint64
r_mhartid()
{
    uint64 x;
    asm volatile("csrr %0, mhartid" : "=r"(x));
    return x;
}

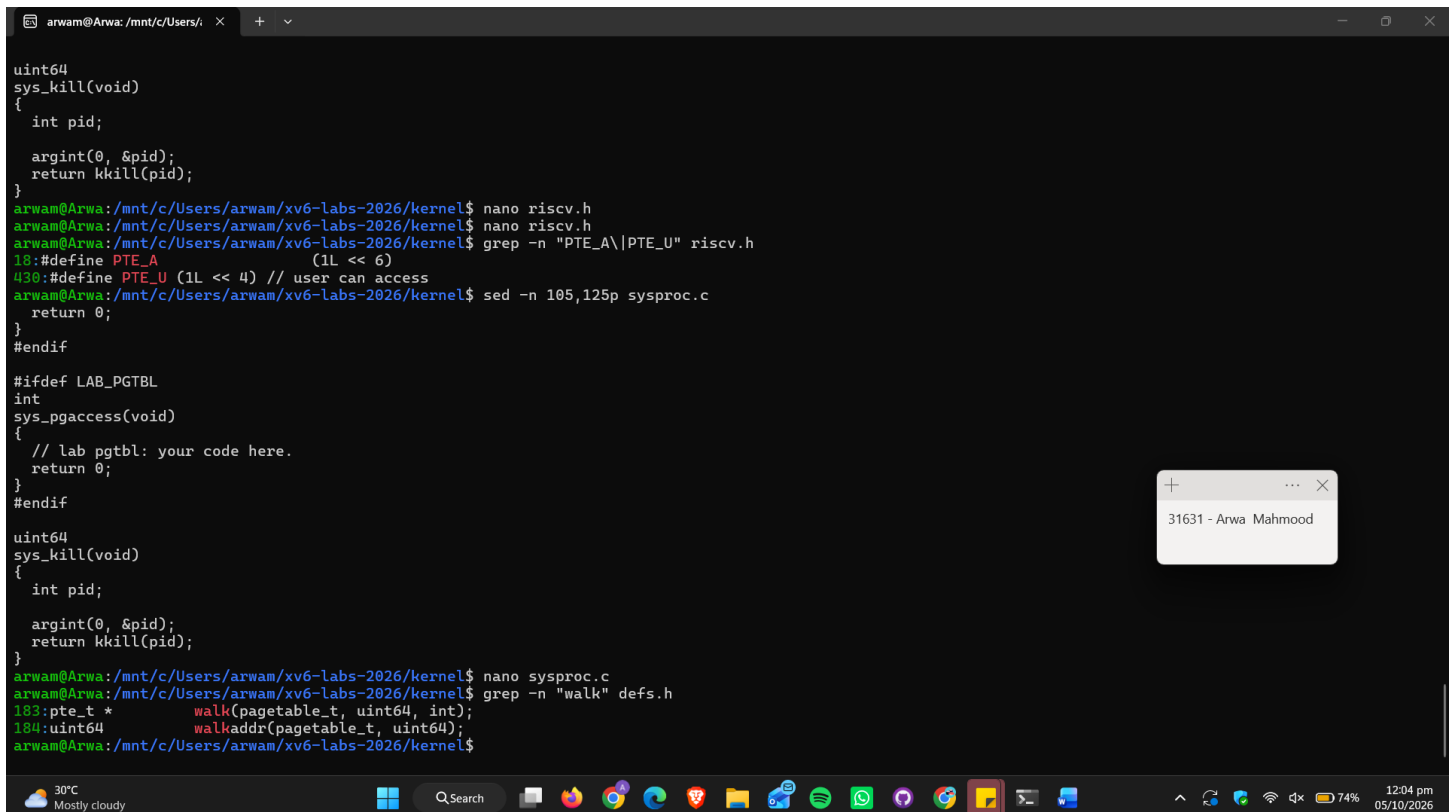
// Machine Status Register, mstatus

#define MSTATUS_MPP_MASK (3L << 11) // previous mode.
#define MSTATUS_MPP_M (3L << 11)
#define MSTATUS_MPP_S (1L << 11)
#define MSTATUS_MPP_U (0L << 11)
#define PTE_A (1L << 6)

static inline uint64
r_mstatus()
{
    uint64 x;
    asm volatile("csrr %0, mstatus" : "=r"(x));
    return x;
}

static inline void
w_mstatus(uint64 x)
{
    asm volatile("csw mstatus, %0" : : "r"(x));
}

// machine exception program counter, holds the
// instruction address to which a return from
// exception will go.
static inline void
```



The screenshot shows a terminal window with the nano editor open to `sys_kill(void)` in `sysproc.c`. Below the editor, several terminal commands are executed to search for `PTE_A` and `walk` in the kernel source. A notification bubble in the top right corner reads "31631 - Arwa Mahmood". The terminal's status bar at the bottom shows the temperature as 30°C, weather as "Mostly cloudy", and the date/time as 12:04 pm on 05/10/2026.

```
uint64
sys_kill(void)
{
    int pid;

    argint(0, &pid);
    return kkill(pid);
}

arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ nano riscv.h
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ nano riscv.h
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ grep -n "PTE_A\|PTE_U" riscv.h
18:#define PTE_A (1L << 6)
430:#define PTE_U (1L << 4) // user can access
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ sed -n 105,125p sysproc.c
    return 0;
}
#endif

#ifdef LAB_PGTLB
int
sys_pgaccess(void)
{
    // lab pgtbl: your code here.
    return 0;
}
#endif

uint64
sys_kill(void)
{
    int pid;

    argint(0, &pid);
    return kkill(pid);
}

arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ nano sysproc.c
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$ grep -n "walk" defs.h
183:pte_t * walk(pagetable_t, uint64, int);
184:uint64 walkaddr(pagetable_t, uint64);
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026/kernel$
```


Q4. Speed up the getpid system call (easy)

```
arwam@Arwa: /mnt/c/Users/...  
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B  
va 0x1000 pte 0x21FC7017 pa 0x87F1C000 perm 0x17  
va 0x2000 pte 0x21FC6C07 pa 0x87F1B000 perm 0x7  
va 0x3000 pte 0x21FC6807 pa 0x87F1A000 perm 0xD7  
va 0x4000 pte 0x0 pa 0x0 perm 0x0  
va 0x5000 pte 0x0 pa 0x0 perm 0x0  
va 0x6000 pte 0x0 pa 0x0 perm 0x0  
va 0x7000 pte 0x0 pa 0x0 perm 0x0  
va 0x8000 pte 0x0 pa 0x0 perm 0x0  
va 0x9000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFF6000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFF7000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFF8000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFF9000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFFA000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFFB000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFFC000 pte 0x0 pa 0x0 perm 0x0  
va 0x3FFFFFD000 pte 0x21FD0413 pa 0x87F52000 perm 0x13  
va 0x3FFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7  
va 0x3FFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B  
print_ptbl: OK  
vmprint_test starting  
page table 0x0000000087f22000  
...0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000  
...0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000  
...0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000 RXU  
...0x0000000000001000: pte 0x0000000021fc7017 pa 0x0000000087f1c000 RWU  
...0x0000000000002000: pte 0x0000000021fc6c07 pa 0x0000000087f1b000 RW  
...0x0000000000003000: pte 0x0000000021fc68d7 pa 0x0000000087f1a000 RWU  
...0x000000003fc000000: pte 0x0000000021fc8401 pa 0x0000000087f21000  
...0x000000003fffe0000: pte 0x0000000021fc8001 pa 0x0000000087f20000  
...0x000000003fffffd000: pte 0x0000000021fd4813 pa 0x0000000087f52000 RU  
...0x000000003fffffe000: pte 0x0000000021fd00c7 pa 0x0000000087f40000 RW  
...0x000000003fffff000: pte 0x0000000020001c4b pa 0x0000000080007000 RX  
vmprint_test: done  
pgaccess_test starting  
pgaccess_test: OK  
#getpid_test starting  
#getpid_test: OK  
pgtbltest: ksuper_test failed: not enough of super ptes, pid=3  
$
```

Q. Which other xv6 system call(s) could be made faster using this shared page? Explain how.

Ans. 'getppid' is the best fit. Add an int ppid field to struct usyscall and let user code read it. The kernel must keep it correct. Set it in fork for the child. Update it in reparent() when a parent exits, because the child's parent becomes init.

- o uptime also works. Map one read-only page that all processes share, and let the timer interrupt (clockintr) write the tick count into it. User code then reads ticks directly.

The reason why its faster: a normal system call needs a trap into the kernel. That means saving registers, switching page tables through the trampoline, running the handler, and returning. Reading the shared page is one memory load. This only suits calls that read values the kernel already has and that have no side effects.

Q5. Use superpages (easy)

```
arwam@Arwa: /mnt/c/Users/i/ + ▼
$ echo Arwa Mahmood ERP 31631
Arwa Mahmood ERP 31631
$ pgtbltest
print_pttbl starting
va 0x0 pte 0x21FE385B pa 0x87F8E000 perm 0x5B
va 0x1000 pte 0x21FE4417 pa 0x87F91000 perm 0x17
va 0x2000 pte 0x21FE4807 pa 0x87F92000 perm 0x7
va 0x3000 pte 0x21FE4CD7 pa 0x87F93000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x21FD013 pa 0x87F7C000 perm 0x13
va 0x3FFFFFFE000 pte 0x21FD4C7 pa 0x87F7D000 perm 0xC7
va 0x3FFFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
print_pttbl: OK
vmprint_test starting
page table 0x0000000087f8b000
..0x0000000000000000: pte 0x0000000021fe3c01 pa 0x0000000087f8f000
.. ..0x0000000000000000: pte 0x0000000021fe4001 pa 0x0000000087f90000
.. .. ..0x0000000000000000: pte 0x0000000021fe385b pa 0x0000000087f8e000 RXU
.. .. ..0x00000000000001000: pte 0x0000000021fe4417 pa 0x0000000087f91000 RWU
.. .. ..0x00000000000002000: pte 0x0000000021fe4807 pa 0x0000000087f92000 RW
.. .. ..0x00000000000003000: pte 0x0000000021fe4cd7 pa 0x0000000087f93000 RWU
.. ..0x0000003fc0000000: pte 0x0000000021fe3001 pa 0x0000000087f8c000
.. ..0x00000003ffffe0000: pte 0x0000000021fe3401 pa 0x0000000087f8d000
.. .. ..0x00000003ffffd000: pte 0x0000000021fd013 pa 0x0000000087f7c000 RU
.. .. ..0x00000003ffffc000: pte 0x0000000021fd4c7 pa 0x0000000087f7d000 RW
.. .. ..0x00000003ffffb000: pte 0x0000000020001c4b pa 0x0000000080007000 RX
vmprint_test: done
pgaccess_test starting
pgaccess_test: OK
ugetpid_test starting
ugetpid_test: OK
ksuper_test: OK
pgtbltest: all tests succeeded
```

```
1 file changed, 0 insertions(+), 0 deletions(-)
arwam@Arwa: /mnt/c/Users/arwam/xv6-labs-2026$ grep -n "kvmmake" -A 22 kernel/vm.c
22:kvmmake(void)
23-{
24-    pagetable_t kpgtbl;
25-
26-    kpgtbl = (pagetable_t)kalloc();
27-    memset(kpgtbl, 0, PGSIZE);
28-
29-    // uart registers
30-    kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
31-
32-    // virtio mmio disk interface
33-    kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);
34-
35-#ifdef LAB_NET
36-    // PCI-E ECAM (configuration space), for pci.c
37-    kvmmap(kpgtbl, 0x30000000L, 0x30000000L, 0x10000000, PTE_R | PTE_W);
38-
39-    // pci.c maps the e1000's registers here.
40-    kvmmap(kpgtbl, 0x40000000L, 0x40000000L, 0x20000, PTE_R | PTE_W);
41-#endif
42-
43-#ifdef LAB_LOCK
44-    // CLINT
45-
46-    kernel_pagetable = kvmmake();
47-}
48-
49-// Switch the current CPU's h/w page table register to
50-// the kernel's page table, and enable paging.
51-void
52-kvmminithart()
53-{
54-    // wait for any previous writes to the page table memory to finish.
55-    sfence_vma();
56-
57-    w_satp(MAKE_SATP(kernel_pagetable));
58-
59-    // flush stale entries from the TLB.
60-    sfence_vma();
61-}
```

31631 - Arwa Mahmood

```
arwam@Arwa: /mnt/c/Users/ / x + v
va 0x0 pte 0x21FDF85B pa 0x87F7E000 perm 0x58
va 0x1000 pte 0x21FDEC17 pa 0x87F7B000 perm 0x17
va 0x2000 pte 0x21FDE807 pa 0x87F7A000 perm 0x7
va 0x3000 pte 0x21FDE4D7 pa 0x87F79000 perm 0x07
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x21FEC413 pa 0x87FB1000 perm 0x13
va 0x3FFFFFFE000 pte 0x21FE7CC7 pa 0x87F9F000 perm 0xC7
va 0x3FFFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
print_pgtbl: OK
vmprint_test starting
page table 0x0000000087f81000
..0x0000000000000000: pte 0x0000000021fdf401 pa 0x0000000087f7d000
..0x0000000000000000: pte 0x0000000021fdf001 pa 0x0000000087f7c000
..0x0000000000000000: pte 0x0000000021fdf85b pa 0x0000000087f7e000 RXU
..0x0000000000001000: pte 0x0000000021fdec17 pa 0x0000000087f7b000 RWU
..0x0000000000002000: pte 0x0000000021fde807 pa 0x0000000087f7a000 RW
..0x0000000000003000: pte 0x0000000021fde4d7 pa 0x0000000087f79000 RWU
..0x00000003fc000000: pte 0x0000000021fe0001 pa 0x0000000087f80000
..0x00000003ffff0000: pte 0x0000000021fdfc01 pa 0x0000000087f7f000
..0x00000003fffffd000: pte 0x0000000021fec413 pa 0x0000000087fb1000 RU
..0x00000003fffffe000: pte 0x0000000021fe7cc7 pa 0x0000000087f9f000 RW
..0x00000003ffffff000: pte 0x0000000020001c4b pa 0x0000000080007000 RX
vmprint_test: done
pgaccess_test starting
pgaccess_test: OK
ugetpid_test starting
ugetpid_test: OK
ksuper_test: OK
pgtbltest: all tests succeeded
```

```
GNU nano 7.2 kernel/vm.c
kmappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm)
{
    uint64 end = va + size;

    if((va % PGSIZE) != 0)
        panic("kmappages: va not aligned");
    if((size % PGSIZE) != 0)
        panic("kmappages: size not aligned");
    if(size == 0)
        panic("kmappages: size");

    while(va < end){
        uint64 step;
        if(va % SUPERPGSIZE == 0 && pa % SUPERPGSIZE == 0 && end - va >= SUPERPGSIZE){
            // superpage: write a leaf PTE at level 1
            pagetable_t pt = pagetable;
            pte_t *pte = &pt[PX(2, va)];
            if(*pte & PTE_V){
                pt = (pagetable_t)PTE2PA(*pte);
            } else {
                if((pt = (pagetable_t)kalloc()) == 0)
                    return -1;
                memset(pt, 0, PGSIZE);
                *pte = PA2PTE(pt) | PTE_V;
            }
            pte = &pt[PX(1, va)];
            if(*pte & PTE_V)
                panic("kmappages: remap");
            *pte = PA2PTE(pa) | perm | PTE_V;
            nsuper++;
            step = SUPERPGSIZE;
        } else {
            pte_t *pte = walk(pagetable, va, 1);
            if(pte == 0)
                return -1;
            if(*pte & PTE_V)
                panic("kmappages: remap");
        }
    }
}
```


Each superpage maps 2 MB directly, so it replaces a bottom-level page table of 512 PTEs ($512 \times 4 \text{ KB} = 2 \text{ MB}$).

Saved: $95 \times 512 = 48,640$ bottom-level pages (PTEs), which is 95 bottom-level page-table pages, or $95 \times 4 \text{ KB} = 380 \text{ KB}$ of page-table memory. It also reduces TLB misses, since one TLB entry covers 2 MB instead of 4 KB.